

Symbolic Execution is (Not Quite) All You Need

Sophie Smithburg

Abstract—We present a technique for extracting verified dispatch automata from ELF binaries via symbolic execution and partition refinement. The extracted labeled transition system is bisimilar to the concrete dispatch behavior, proved in Lean 4 with no admitted axioms. Guards and updates are expressed in the quantifier-free bitvector fragment of SMT-LIB, enabling automated vulnerability detection via Datalog queries. In a blind trial, a 119-line Soufflé program correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375 in mjs v1.20.1 from anonymized dispatch facts alone.

Index Terms—symbolic execution, formal verification, labeled transition systems, partition refinement, LangSec, Lean 4, Datalog

I. INTRODUCTION

No one writes specs; everyone writes implementations. But you can’t prove things about implementations directly—you can about specs. So in order to have an effective ratchet in the sense of Metzger’s LangSec keynote [1]—addressing the specification-implementation gap [2]—we want to be able to get specs from implementations.

We can model a programming language executing on a host machine as a host labeled transition system H [3] which, after having been fed $\mathcal{I}$, the implementation, simulates G —the PL is G , its implementation is $\mathcal{I}$, and the host machine is H . To prove a recovered G' faithfully captures G , we would need G in a formalism that shares the same proof basis as G' —but in practice, this is never the case. So what we do instead is prove that G' simulates $H_{\mathcal{I}}$, by way of projecting away dispatch-irrelevant state and transitions.

We have implemented a technique and have an early proof of concept of a system that, by way of symbolic execution and Paige-Tarjan partition refinement [4], can extract a strongly bisimilar symbolic labeled transition system from an ELF binary—guards and updates expressed in the quantifier-free bitvector fragment of SMT-LIB. We have successfully used this to generate vulnerability detection queries in Soufflé [5], a Datalog dialect oriented towards static analysis. In a blind trial, Claude Opus 4.6 was given only two toy synthetic examples of a vulnerability class, wrote a 119-line Soufflé program that distinguished bounded from unbounded recursion, and when applied blind to anonymized facts from mjs v1.20.1, correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375. We also used the technique to confirm the absence of certain terminal emulator vulnerability classes in libtasm, by passing a description of the vulnerability classes along with the extracted LTS to an LLM agent.

This paper and its Lean 4 mechanization were developed with extensive use of Claude Code and ChatGPT, some use of Codex CLI, and a smattering of other LLM tools in earlier prototypes.

REFERENCES

- [1] P. E. Metzger, “Slow but steady: Achieving real security within two decades,” Keynote, Language-Theoretic Security Workshop (LangSec), IEEE Security and Privacy Workshops, 2017.
- [2] L. Sassaman, M. L. Patterson, and S. Bratus, “Security applications of formal language theory,” *IEEE Security & Privacy*, vol. 11, no. 3, pp. 52–59, 2013.
- [3] T. A. Henzinger, R. Majumdar, and J.-F. Raskin, “A classification of symbolic transition systems,” in *Proceedings of STACS 2000*, ser. Lecture Notes in Computer Science, vol. 1770. Springer, 2000, pp. 13–34.
- [4] R. Paige and R. E. Tarjan, “Three partition refinement algorithms,” *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.
- [5] H. Jordan, B. Scholz, and P. Subotić, “Soufflé: On synthesis of program analyzers,” in *Computer Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 9780. Springer, 2016, pp. 422–430.